



ポートフォリオ

五十嵐 幸太

ユーザー・透視投影
(1) 目玉焼き | フライパン <F_01>

オブジェクト 29
頂点 14,947
辺 29,867
面 15,072
三角形面 25,730

自己紹介

Igarashi kota
五十嵐 幸太

生年月日 2003年8月20日
生まれ 神奈川県相模原市
趣味 ゲーム、テニス、紅茶
好きなゲーム スプラトゥーンシリーズ、Balatro
所属 成蹊大学 理工学部
スマートニューロリハビリテーション研究室
美術部
相模原ローターアクトクラブ

使用アプリケーション



blender (3DCG)
3Dサブスタンスペインター (3DCG)
Fusion360 (CAD)
Creo Parametric (CAD)



Visual Studio Code (プログラミング)
Visual Studio (プログラミング)
Jupyter Notebook (プログラミング)
Node Red (ノードプログラミング)

スキル

Python ★★★★★
C++ ★★★★★
C# ☆★★★★
HTML/CSS/Javascript ☆☆☆★★
ポリゴンモデリング ★★★★★
ソリッドモデリング(CAD) ★★★★★
スカルプトモデリング ☆☆☆☆★

このポートフォリオに関して

成果物のタイプごとに4つの章に分かれており、各章は
1.ゲーム開発、2.3DCG、3.プログラミング、4.その他 という項目です。

基本的には1ページ1作品の構成となっています。例外として、最初の
ゲーム開発の章に関しては章全体が1ゲーム作品分となっています。逆に、
その他の章などは複数作品がまとまって構成されているページもあります。
ざっと見るだけでもおおまかな内容は把握できると思いますので、是非
気軽に見て頂けるとありがたいです。

また作品および章は、時系列順ではなく見て頂きたい順に並んでいます。

作品画像の中にはクリックでyoutubeに遷移できるものもあります。
セキュリティの都合上危ない場合はハイパーリンクを削除した
バージョンもございますので、その場合はそちらをご覧頂けると幸いです。

登場するハイパーリンク一覧

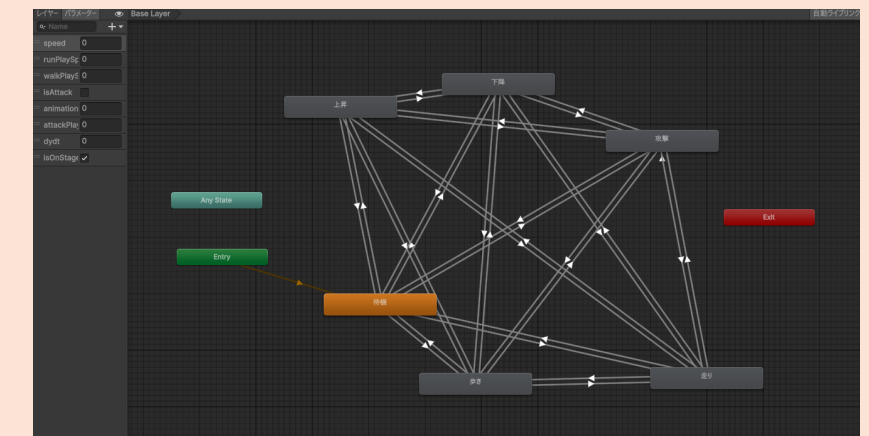
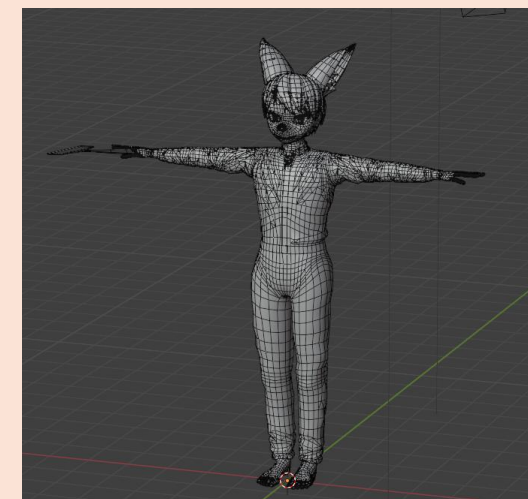
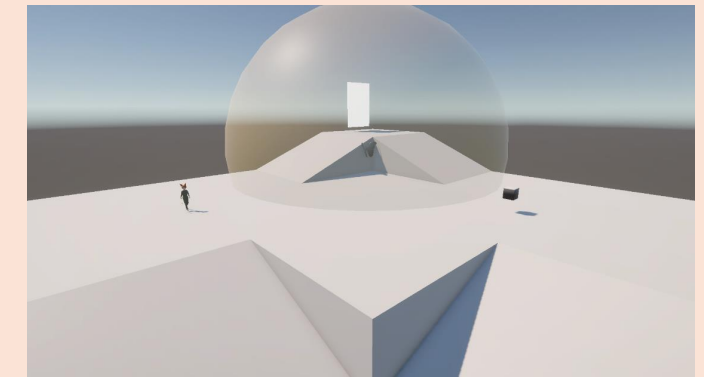
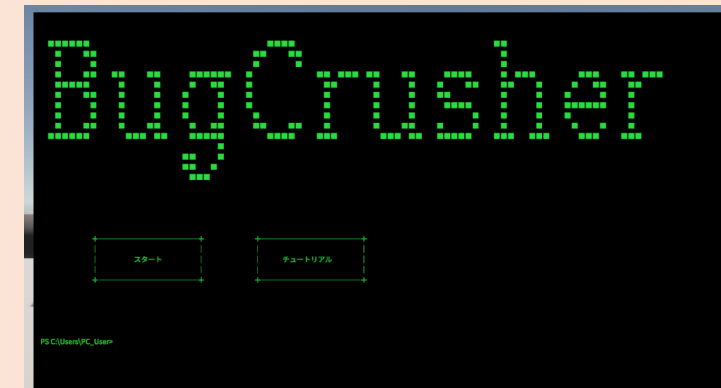
- ・ ゲームのソースコードのGitのリンク (<https://github.com/igarashikota/BugCrusher>)
- ・ youtubeへの動画リンク (<https://youtu.be/kClBaVV66x0>)

ゲーム開発

3DCD

プログラミング

その他 (CADと絵)



▲ここをクリックでGitHubに移動し、ソースコードや実行ファイルがご覧いただけます。
(<https://github.com/igarashikota/BugCrusher>)

全6ページ

ゲーム全体像

主人公はこのゲームの制作者です。ひとまずゲームは完成しましたが、いざ実行してみるとバグだらけ。幾何学的な本来のゲームの世界をクリアしていきつつ、この世界に実体化したバグたちを潰してゲームを完成させましょう！

自身を様々な方向に強化できる
ショップもあるほか、開発者ツール
を使って異次元な強化も・・・？



2025年6月

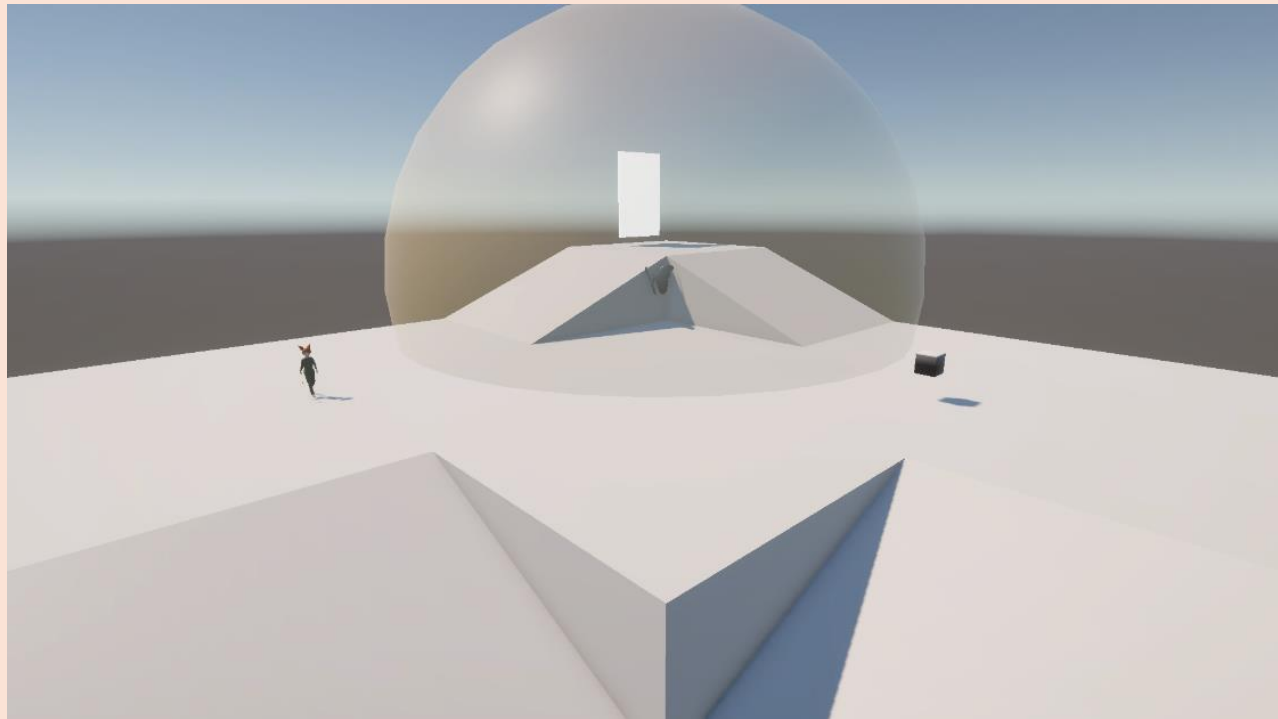
Unityを用いて作成しているゲーム、「BugCrusher」です。デザインやモデリング、プログラミングを作成しました。

こだわったポイント

自分の好きな、「ローグライク」「TPS」「メタ的要素がある」といった要素を詰め込んでいます。ゲームバランス的には、自身の強化の方向によって、様々なゲーム体験が出来るよう目指しています。

とりあえずゲームとして成り立つように、こだわりたい箇所はこだわり、そうでない部分は最小限の完成度で作成しました。

このスライドで全体像を、次と次のスライドで世界観の説明とプレイ動画。その後、モデリングとプログラミングの詳細スライド×2の計3枚、全てで6枚のスライドでの構成です。



主人公はゲーム開発者で、幾何学的な世界観のゲームを作っています。開発はだいたい終わり、デバッグ作業を残しています。このゲームは、主人公がデバッグ作業をするためにゲームをプレイするところから始まります。

このゲームの目的は、主人公がゲーム内の主人公を動かしながら、実際にゲームをプレイしつつデバッグをしていくということが目標のゲームです。

本来のゲームに登場する幾何学的な敵の他に、虫がモチーフの敵が出現します。

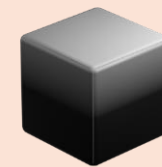
ステージを進めるために本来のゲームの機能で自身を強化しながら、バグが実体化して出現した虫モチーフの敵を退治しましょう。

また、虫たちは厄介な特殊効果を持っているため正攻法ではクリアするのが非常に難しいです。

そのために、開発者ショップ（特殊なショップ）を使ってさらに強力な強化をしつつ進んでいきましょう。

設定

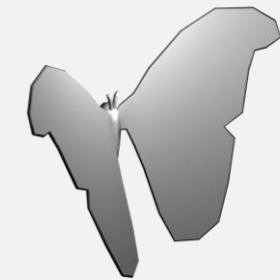
ゲーム本来の敵たち



cube

オーソドックスな敵です。
シンプルに追ってきて、
攻撃してきます。
大抵大量に沸きます。

バグ由来の敵たち



moth

例外処理の化身です。mothは「NullReferenceException」を担当しており、近づいてる間色々なステータスを無効にしてくれます。

現時点では攻撃力を0にしていますが、自身のHPも無効になっているので1撃で確殺できます。

ショップ

通常のショップ



ステージクリアで獲得できるコインを使って、体力や移動スピード火力や獲得コイン量を強化できます。強化上限があります。

開発者ショップ



バグを倒すと貰えるトークンを使って実質的に強化上限を突破したり、強い強化ができたりします。

武器



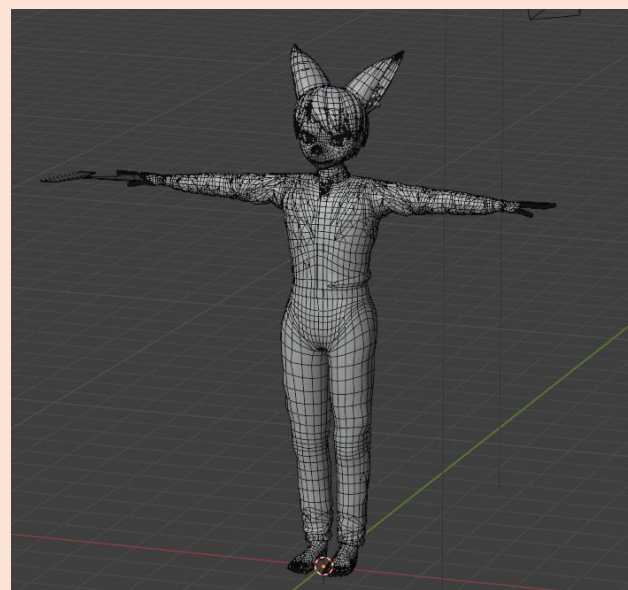
主人公が手にしているのは、ハエたたきです。これでバグをクラッシュしていきます。

いずれ他の武器も持てるようになる予定です。

ゲーム画像



▲動画があります。(02:48) クリックでyoutubeに遷移し動画が見れます。
(https://youtu.be/zR_E1lxpT2s)



△約3万弱

ここからは制作のことを話します。

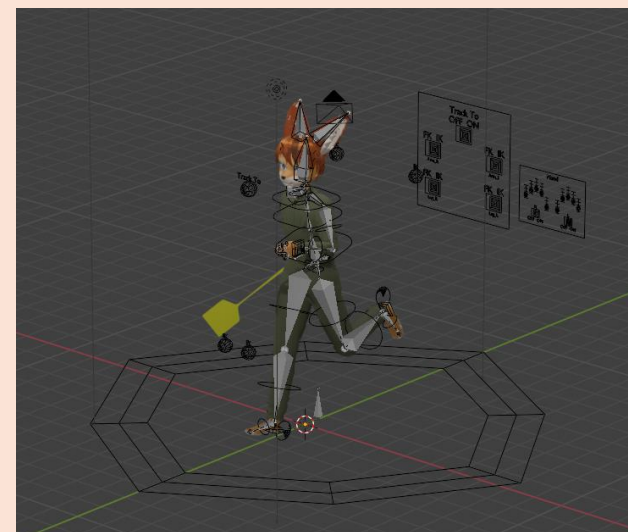
まずモデリングに関してです。主人公はできる範囲でこだわり、一方で敵とステージ、オブジェクトやモーションはある程度の完成度に留めています。

こだわったポイント

モデリングからリギング、初挑戦のアニメーションまで、様々なことを学びながら制作を進めました。またUnityでモデルを動かすということも初めて行いましたが、悪戦苦闘しながらきちんと動くところまでもっていけました。

モデリングでずっと参考にさせて頂いている夏森轄さん、今回ウェイトペイントでKMBLさん、リギングでYonaoshi3Dさん、またHo Yoverseから出ている公式キャラモデルなどを参考にさせて頂きながら、制作を進めていきました。

モデリング



待機、歩き、走り、上昇、下降、攻撃の6モーションを作成しました。服がシンプルということもあり、テクスチャは2KのPNG画像1枚です。

リグと呼ばれる、モデルに骨を与えて動かしやすくする仕組みも、0から作ってみました。

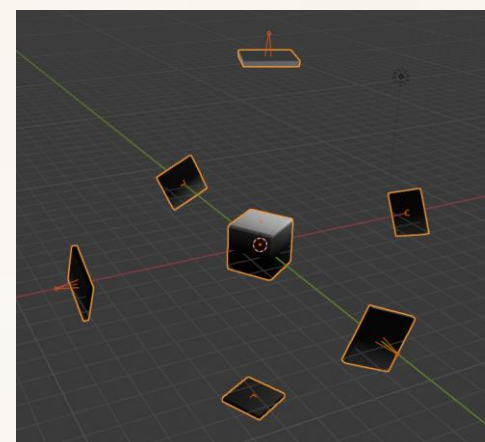
リバースフットリグや腕のロールボーン等を備え、またIKのオンオフもできます。

写真は走っているモーション、テクスチャ、素体(服を着せる前のモデル)です。

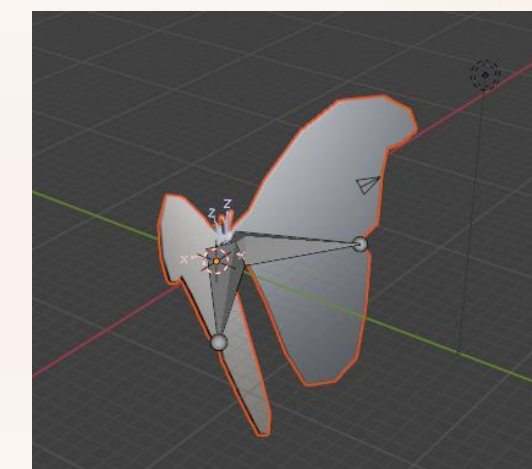
顔のポリゴンの流れ(トポロジー)や工夫は、ゼンゼロの公式から出ているモデルを参考にさせて頂きながら作成しました。

敵の色付けは手で塗っているのではなく、ベイクと呼ばれる処理方法で色を塗っています。

本来はゲームの高速化や色塗りの補助として使われることの多いベイクですが、時短と幾何学的な塗り方ができるということで直接塗りに使っています。



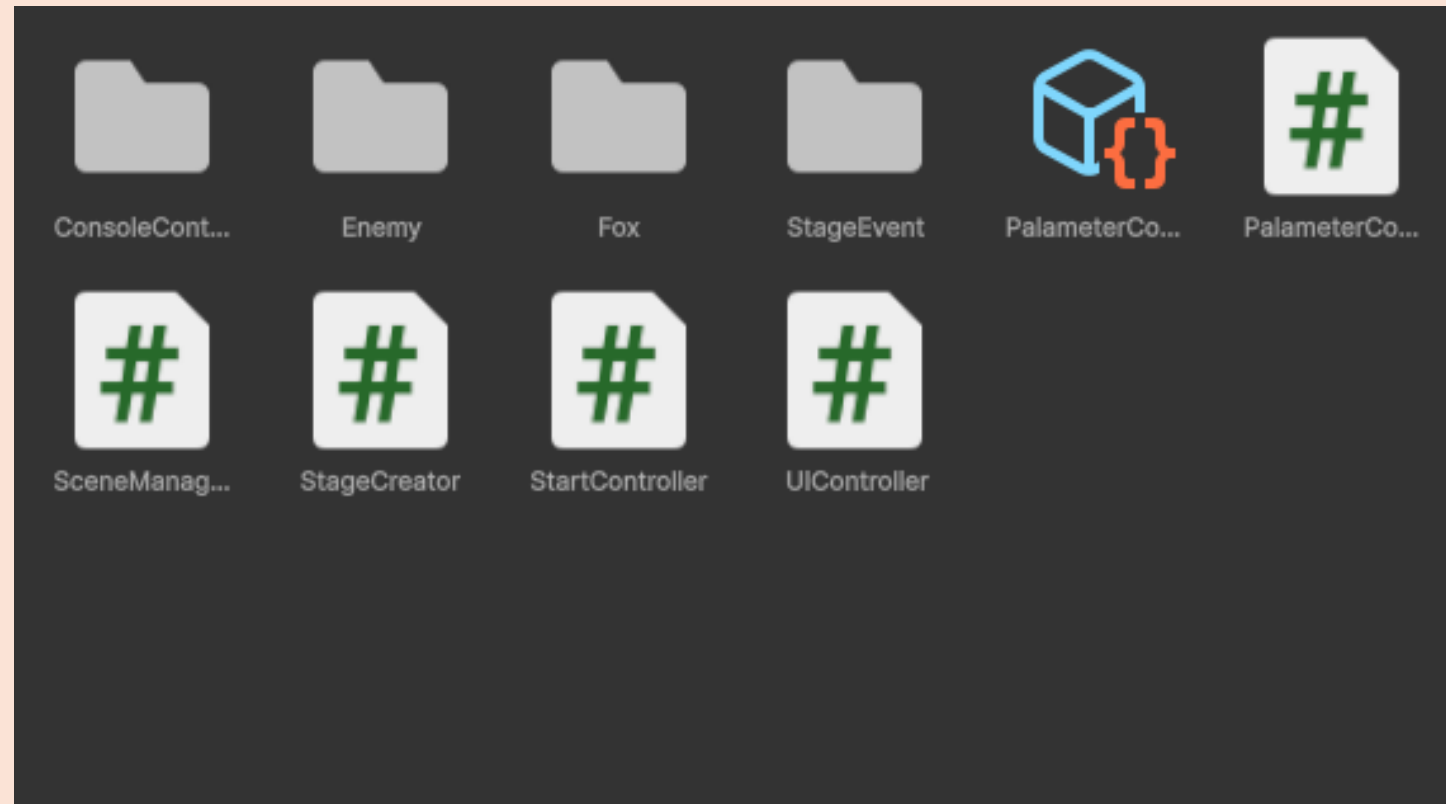
△180



△692

それぞれで待機モーションを持ち、cubeは攻撃モーションを持ちます。こちら簡単なリグで制御しています。

プログラミング(1/2)



プログラミングに関してです。PythonとC++の経験はありましたが、Unityに慣れていなかったのでもここでも苦戦しました。先にUnity公式のチュートリアルや本どおりに進めて習得していきました。

こだわったポイント

しっかりオブジェクト指向を意識し制作しました。

ただし全てを管理するようなスクリプトの整備が不十分で、それぞれのコード間での関数呼び出しが複雑になってしまいました。

作り直すとすれば、もう少し整理されたコードにしたいなと感じます。

プレイヤー制御

[./BugCrusher/Assets/Scripts/Fox/FoxControler.cs](#) や
[./BugCrusher/Assets/3DObjects/Models/Fox/狐.prefab](#) など

狐の獣人です。

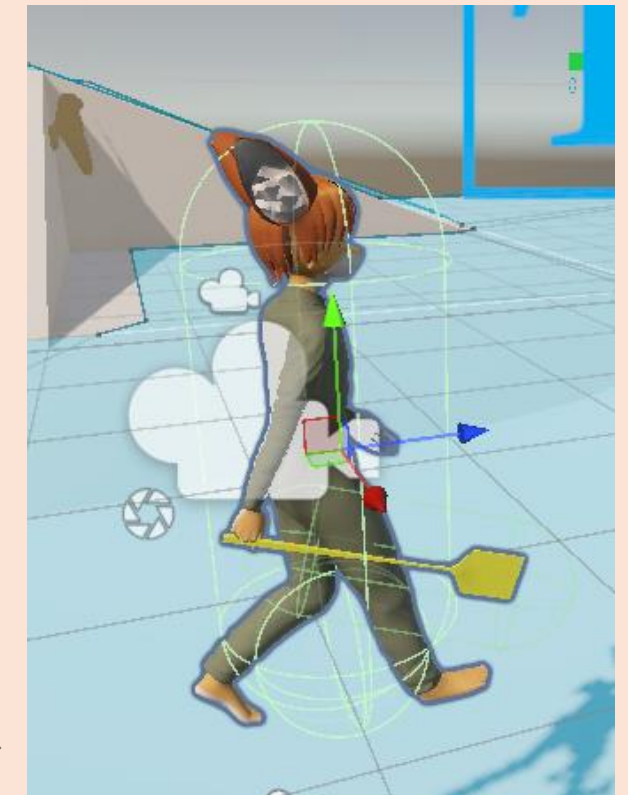
視点はマウスの動きに合わせて、狐の位置座標を参考にしてカメラの向きと位置を動かす方針です。

攻撃判定は武器のボーンに当たり判定を設置しており、攻撃に合わせて有効にしたり大きさを変えています。動き自体はアニメーション任せです。

攻撃したとき/されたときの当たり判定は、すべて狐側から判定します。敵ごとで当たり判定チェックをするわけでは無いので、処理が軽いです。

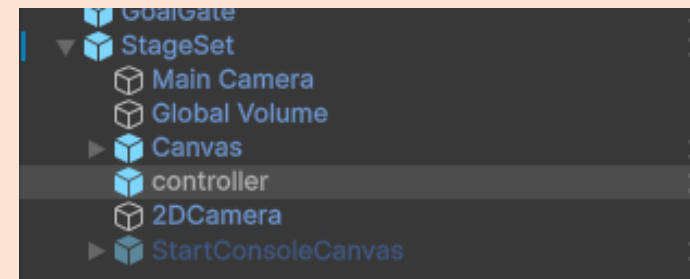
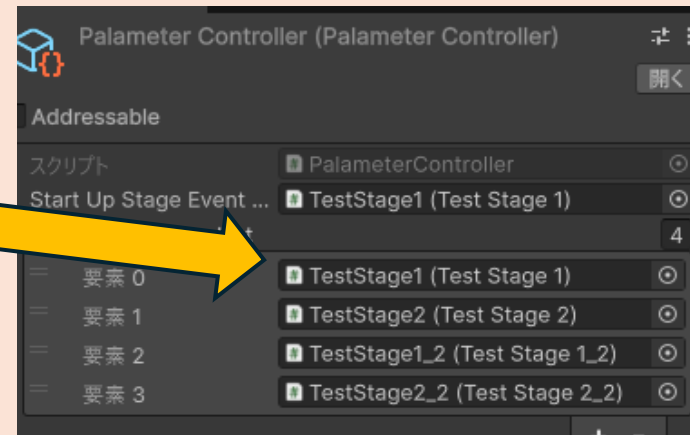
移動速度や攻撃速度などの各パラメータは、ステージ開始時にシーンをまたぐPalameterControllerクラスから読み込み専用の変数で受け取ります。敵も同様で、個体差を持たせたパラメータを、敵がスポーンする際に受け取っています。

敵の「moth」の効果範囲では攻撃が無効になります。これは毎フレームで範囲内か判定しているのではなく、入退場のタイミングでカウントを増減させ判定しています。軽量化目的です。このように少しの軽量化を図った処理を、出来る限りいれています。



プログラミング(2/2)

```
1 using UnityEngine;
2
3 public class TestStage1 : StageEventBase {
4     //private void Awake() {
5     //     _enemyLifeController = GetComponent<EnemyLifeController>();
6     //     _probability = 10000;
7     // }
8
9     private void Start() {
10         int enemyNum = (int)_palameterController.MultiplyLevelMultiplier(5);
11         _enemyLifeController.spawnEnemy("cube", 0, enemyNum);
12         _enemyLifeController.spawnEnemy("cube", 1, 10);
13     }
14
15     public override int GetProbability() {
16         return 10000;
17     }
18
19     public override string stageName {
20         get { return "TestStage"; }
21     }
22
23     public override int clearCoin {
24         get { return 500; }
25     }
26
27     public override string stageDisplayName {
28         get { return "テストステージ1"; }
29     }
30
31     public override string difficulty {
32         get { return "普通"; }
33     }
34
35     public override string rule {
36         get { return "ゲート"; }
37     }
38
39     public override void Clear() {
40         _palameterController.GetCoin(clearCoin);
41     }
42
43
44 }
```



ステージ管理方法

[./BugCrusher/Assets/StageEvent/StageEventBase.cs](#) や、
[./BugCrusher/Assets/StageEvent/Stage/FirstStage.cs](#) など

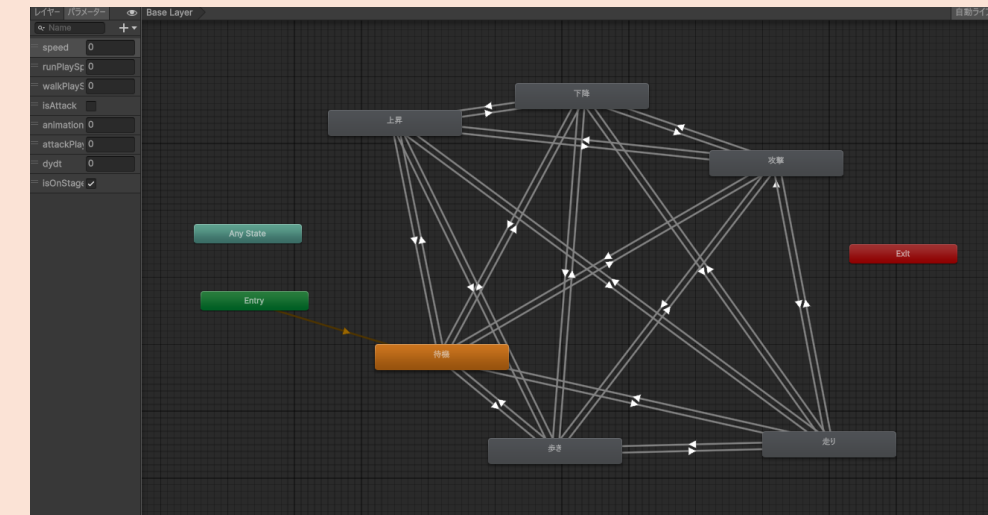
各ステージには「StageEventBase」クラスを継承して、ステージのコントロールをする子クラスが1つずつ存在します。

このスクリプトはステージにあるコントローラオブジェクトにて実体化され、ステージに敵を沸かせたりステージギミックを動かすなどステージの動作を担当します。

そののみならず、実体化していなくても名前や獲得コインなどを返せるようになっており、ステージ選択画面を表示させるためのステージ情報も保持しています。

これにより、ステージの情報やプレイ時の制御がまとめて1つの子クラスで管理できるようになります。

1ステージ1クラス1シーン、プラス共通素材を厳守しているため、ステージが大量に出来ても混乱しにくいシステムになっています。



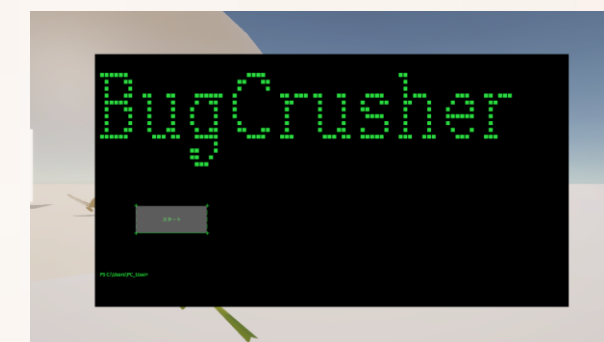
Unityのアニメーターステートマシンという機能でキャラのモーションの移り変わりを管理しています。キー入力や移動速度、落下速度などから適切なモーションに切り替えます。

スクリプトを組んで、移動速度が変わっても足が滑らない工夫なども施しています。



改善点（今後の課題）

このゲームはまだまだ未完成です。世界観を伝えるためにも、全体的な改善が必須です。楽しく遊んで頂くためにシステムやゲームバランスを追加しつつ、カジュアルにも遊べるよう工夫したいと思っています。



改善項目

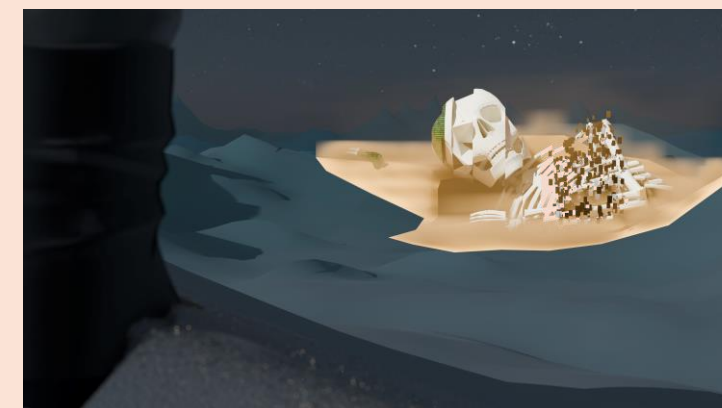
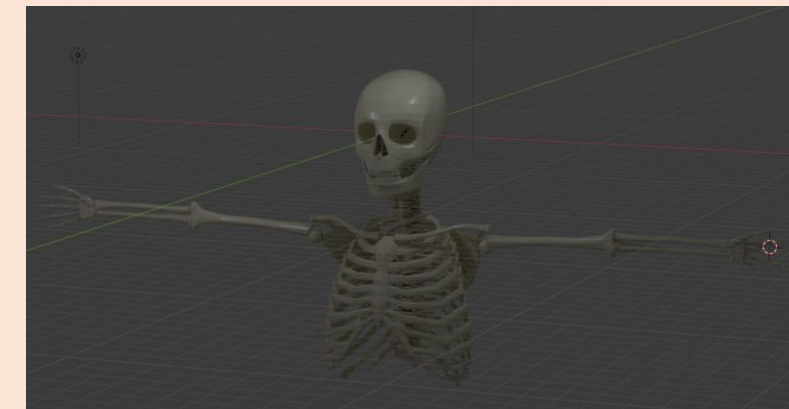
- | | |
|---------|------------|
| • ステージ | (数・外観・ルール) |
| • 敵 | (種類・行動) |
| • 操作キャラ | (外観・行動) |
| • システム | (新規追加・拡充) |
| • バランス | (全体的に修正) |

ゲーム開発

3DCD

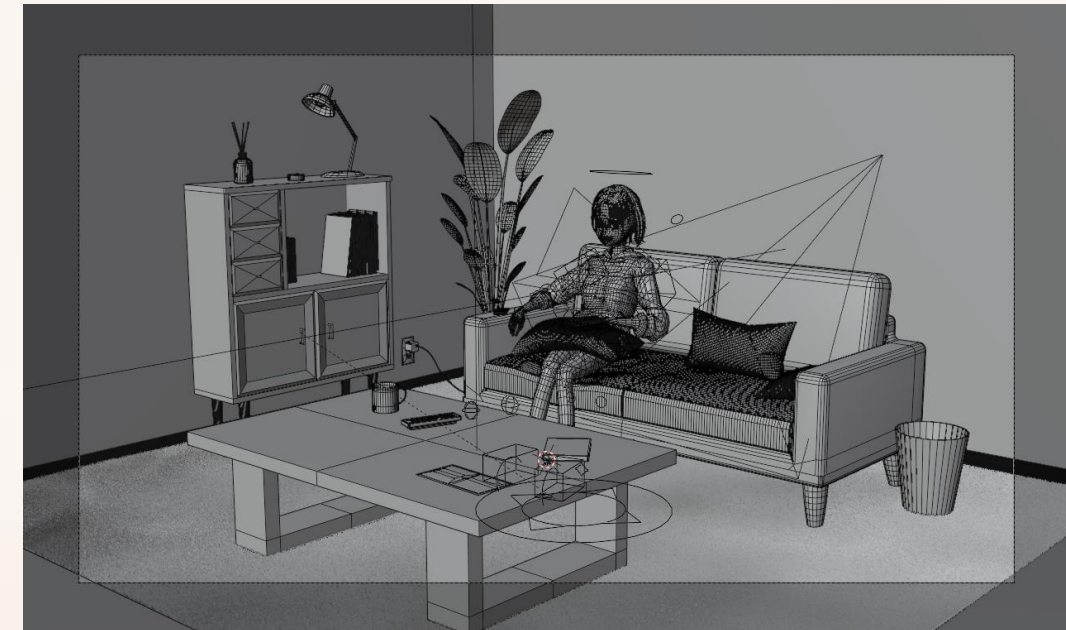
プログラミング

その他
(CADと絵)



全4ページ

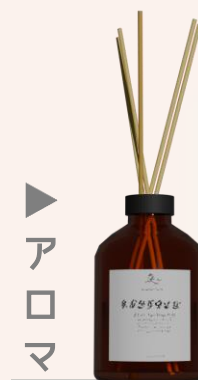
映画鑑賞



▲ワイヤーフレーム



観葉植物



アロマ



▲浅倉さん (Auto Rig Proを使用) ▲



▲明かりがついた状態



▲アイドル衣装の雑誌



blender 2024年7月
制作期間 1か月
△約17万7千

二次創作の浅倉透さんです。
美術部内のイベントで、好きなキャラを交換してお互いに作品を作り合おうということになりました。その際に制作した作品です。

こだわったポイント

アイドルマスターに登場するアイドルですので、逆に日常の1場面を切り取ってみようと思いました。映画や読書が好きらしいので、部屋でのんびりしている姿を制作しました。奥の雑誌にも浅倉さんのアイドル姿の衣装を着せて表紙にしてみました。暗さが原因して見えないのが失敗です。

キャラモデリング自体は3キャラ目でした。顔は本家に寄せきることができていなかったり、皮膚の色が単色であったりなど、改善点は多いです。

小物たちはかなり良い感じにモデリングできたと思います。



blender 2024年11月
制作期間 二週間
△約2万

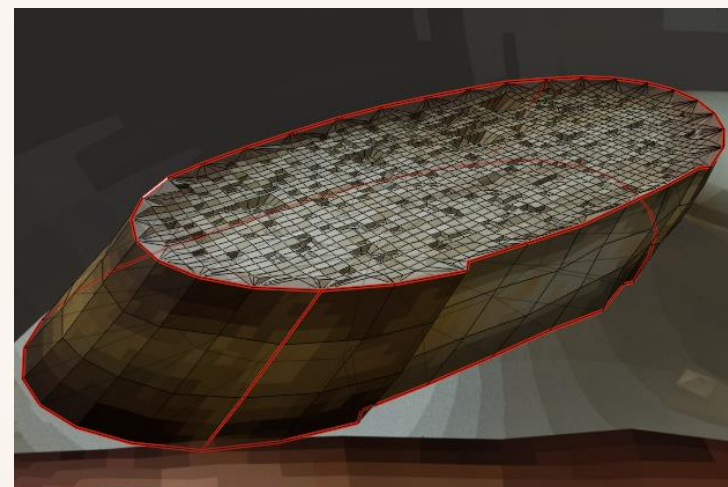
3DCG4作目で、朝食です。文化祭に美術部の展示として出したものです。温かみのある食事を目指し作成しました。

こだわったポイント

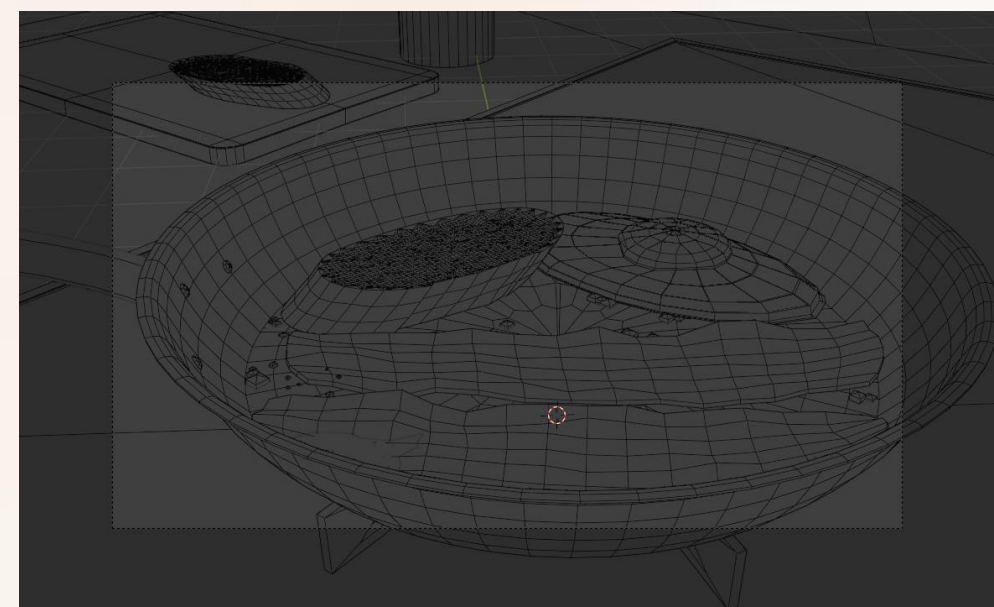
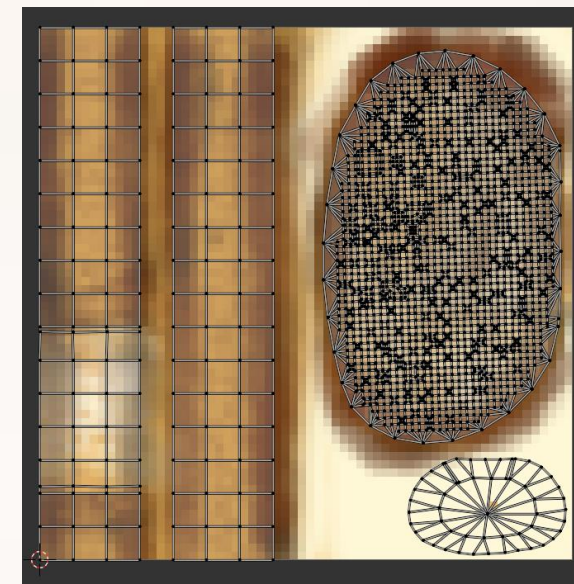
他の美術部員はアナログでの作品が多いということもあり、3DCGでないと難しいローポリかつアニメ風の画風(セルルック)にしてみました。
パンやベーコンの表面も低めの解像度のものを使用し、あえて粗くしています。

また目玉焼きは二重に面を張ることでぷるぷる感を出しています。
フライパンはノードを組んで異方性と呼ばれる特有の金属の反射を作り、フライパンらしさを演出しました。

朝食

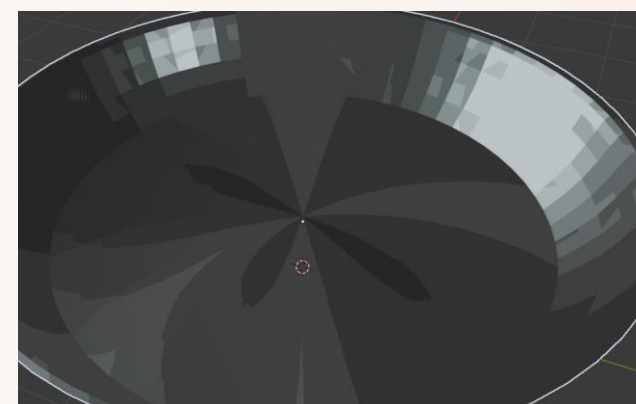


▲▶ フランスパンとそのテクスチャ

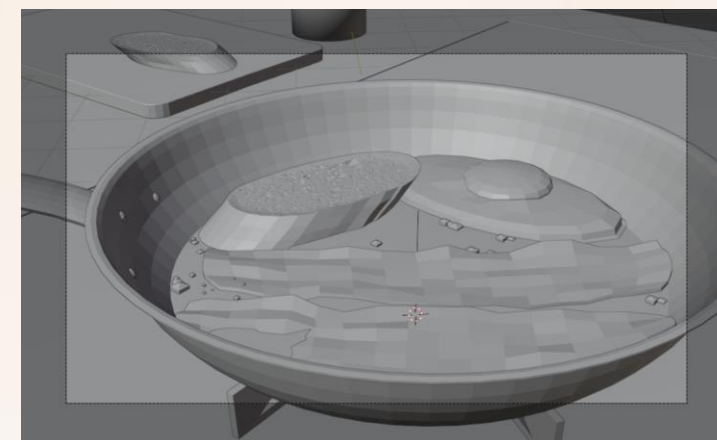


▲ワイヤーフレーム

1つあたり...	△5,414
フランスパン	△385
目玉焼き	△444
ベーコン	△5,186
フライパン	

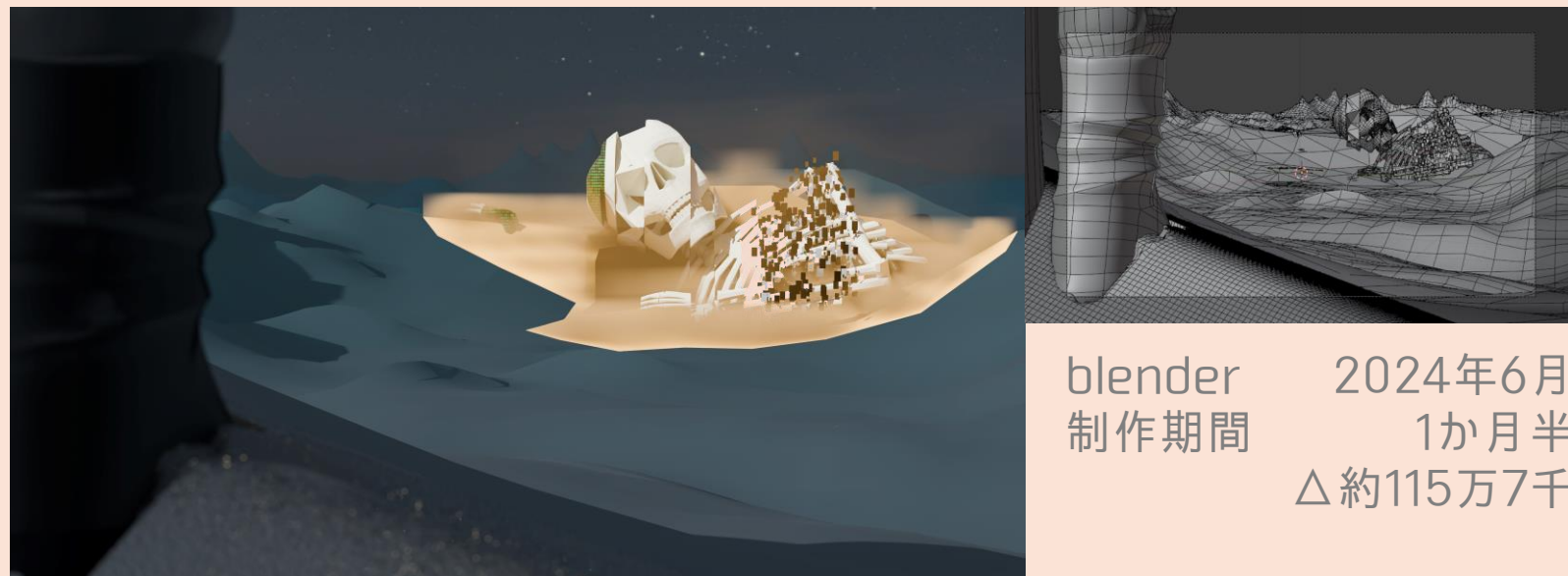


◀異方性



◀ソリッド

雪山



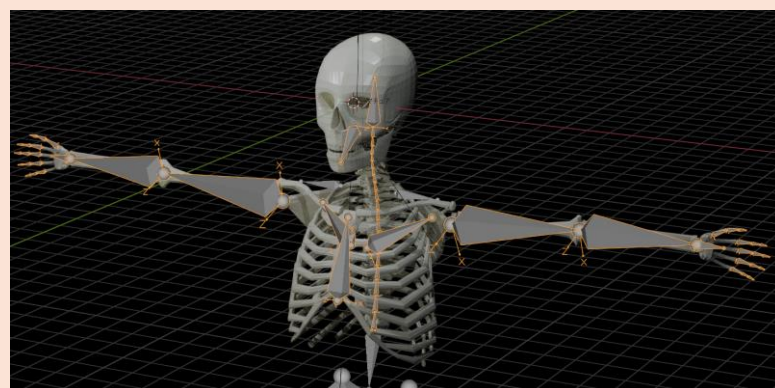
blender 2024年6月
制作期間 1か月半
△約115万7千

CG始めたてのころ、美術部の展示企画で出したものです。
雪の中に、場違いに砂に埋もれているローポリの骸骨があります。

こだわったポイント

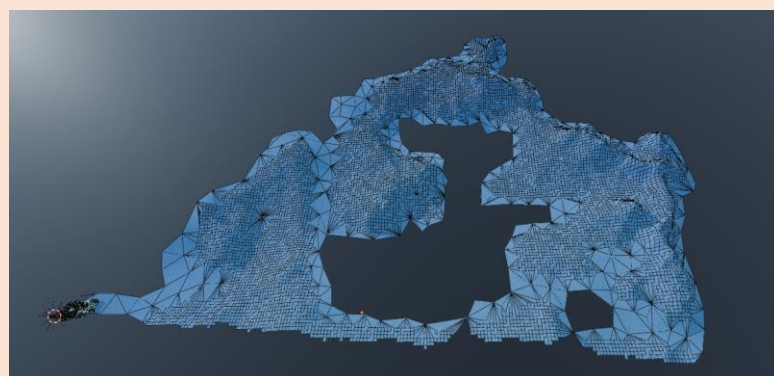
この世のものではない物体を発見したという設定です。
バグった骸骨と周りの空間を色々と対比させています。

骸骨は解剖図や本を見ながらローポリで作り、リグを
仕込みました。



▲ボーン構造です。Unity基準の
ボーンで制御できます。

▼視点との距離で青くし空気遠近法を
再現しています。



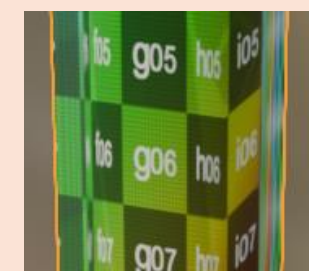
シェーダーの勉強として作成
した視差シェーダーです。
入射ベクトルを使用して、四角柱
を円柱に見せています。

こだわったポイント

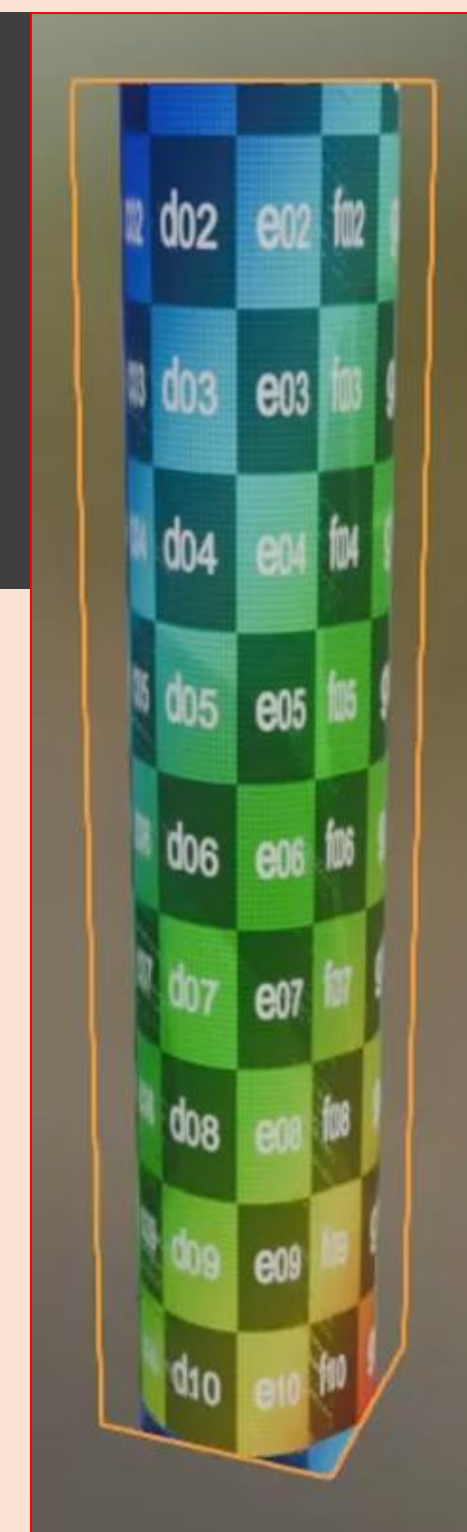
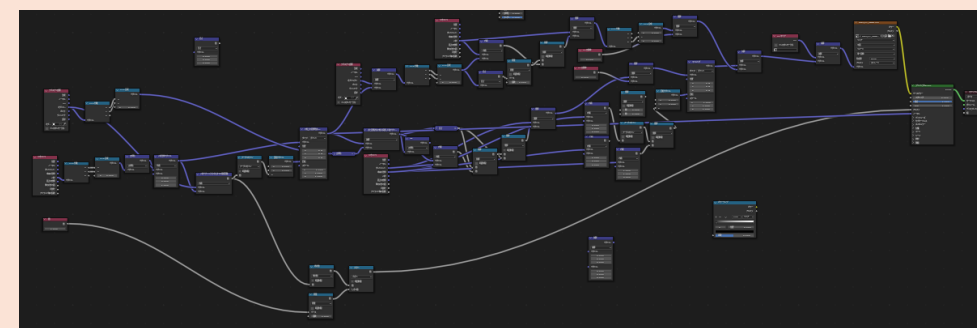
内積や外積などを使用しUV
をズラしています。

視差シェーダー

blender内の3D(x,y,z)の入力で動作する
特殊なノイズテクスチャなどだけではなく、
通常の画像(u,v)で動作するシェーダーを
作成しました。



alpha抜き + U方向補正 + V方向補正 =



▲クリックでyoutubeに
遷移し動画で見れます。
(<https://youtu.be/kCIBaVV66x0?t=160>)

blender 2025年7月
制作期間 2日
△12

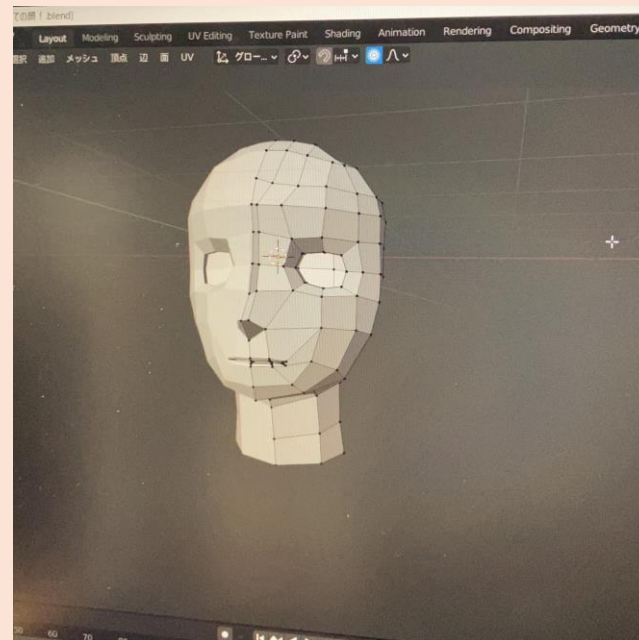
その他



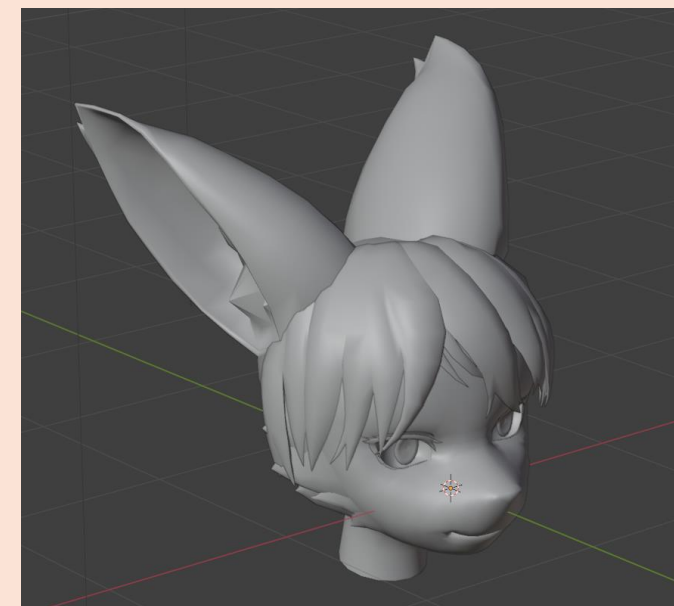
▲blenderで、私が最初に美術部の展示に出した作品です。会場の光が絵の中にさしこんで、現実と絵がリンクしている様子を表したかったのですが、あまりうまく行きませんでした。



▲blenderで本当に最初に作ったものです。本を買ってそれ通りに作ったのですが、今でも愛着があります。



◀▼初めてキャラモデルをしたときのものです。左から下にするために、かなりの時間を費やしました。頭と体が繋がられない事件も発生しました。



▲現在作成している獣人の子です。VRCに対応できるリグやシェーダーも勉強中です。(のちのちゲーム開発で使いました)



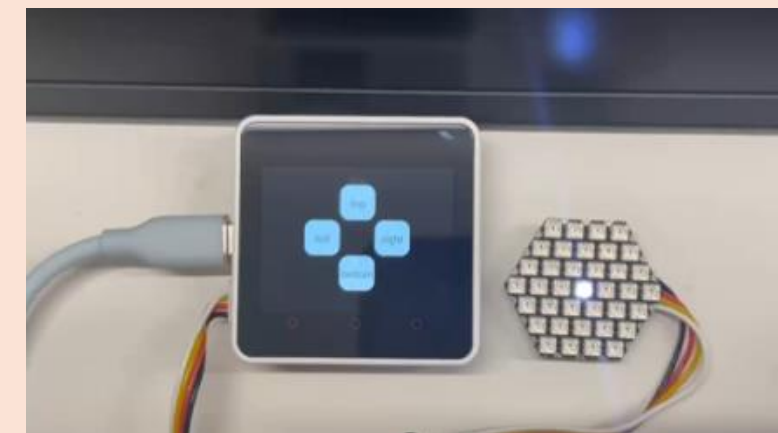
▲二次創作の轟くんです。モデリングに慣れるため、2回目のキャラモデリングとして作りました。

ゲーム開発

3DCD

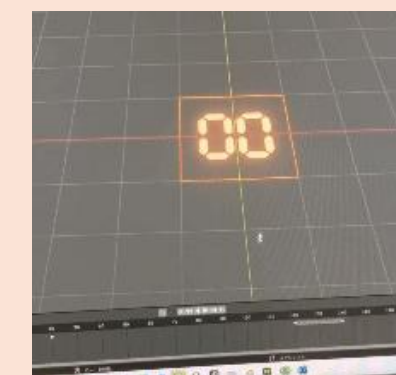
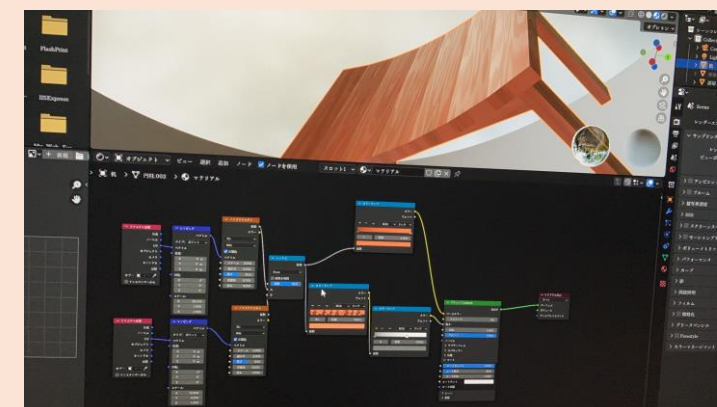
プログラミング

その他
(CADと絵)



 melody.txt

 mel.wav



全4ページ

音楽再生コード

```
#include "music.h"

int main()
{
    music music1;
    music1.filePass = "../melody.txt";
    music1.wavFileName = "mel.wav";
    music1.autoplay();
}
```

VisualStudio 2025年3月4日
制作期間 C++

音楽を作成するコードです。C++で作成しました。応用Pythonプログラミングの小課題を作っている間に作成したものを、かなり改良しました。

今回は、動作確認として「ム責任集合体」(誤字ではないです)という曲のサビの一部を譜面に起こしてくださった方の楽譜を参考にさせて頂き、一定の規則に従ってtxtファイルに入れています。

こだわった点

このコードで初めてオブジェクト指向というものを意識してコードを作りました。複数人でコーディングをする際かなり重要になると聞き、練習の為に作成しました。

扱う際には↑の計5行のコードのみを書くことで、楽譜ファイル読み込みから音楽ファイル作成、再生まで一律で行ってくれます。

00000000	52 49 46 46 A2 6C 1C 00 57 41 56 45 66 6D 74 20	RIFF「I..WAVEfmt
00000010	10 00 00 00 01 00 01 00 44 AC 00 00 88 58 01 00	Dy.. *
00000020	02 00 10 00 64 61 74 61 2C 6C 1C 00 00 00 00 00	data,l.....
00000030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00000040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

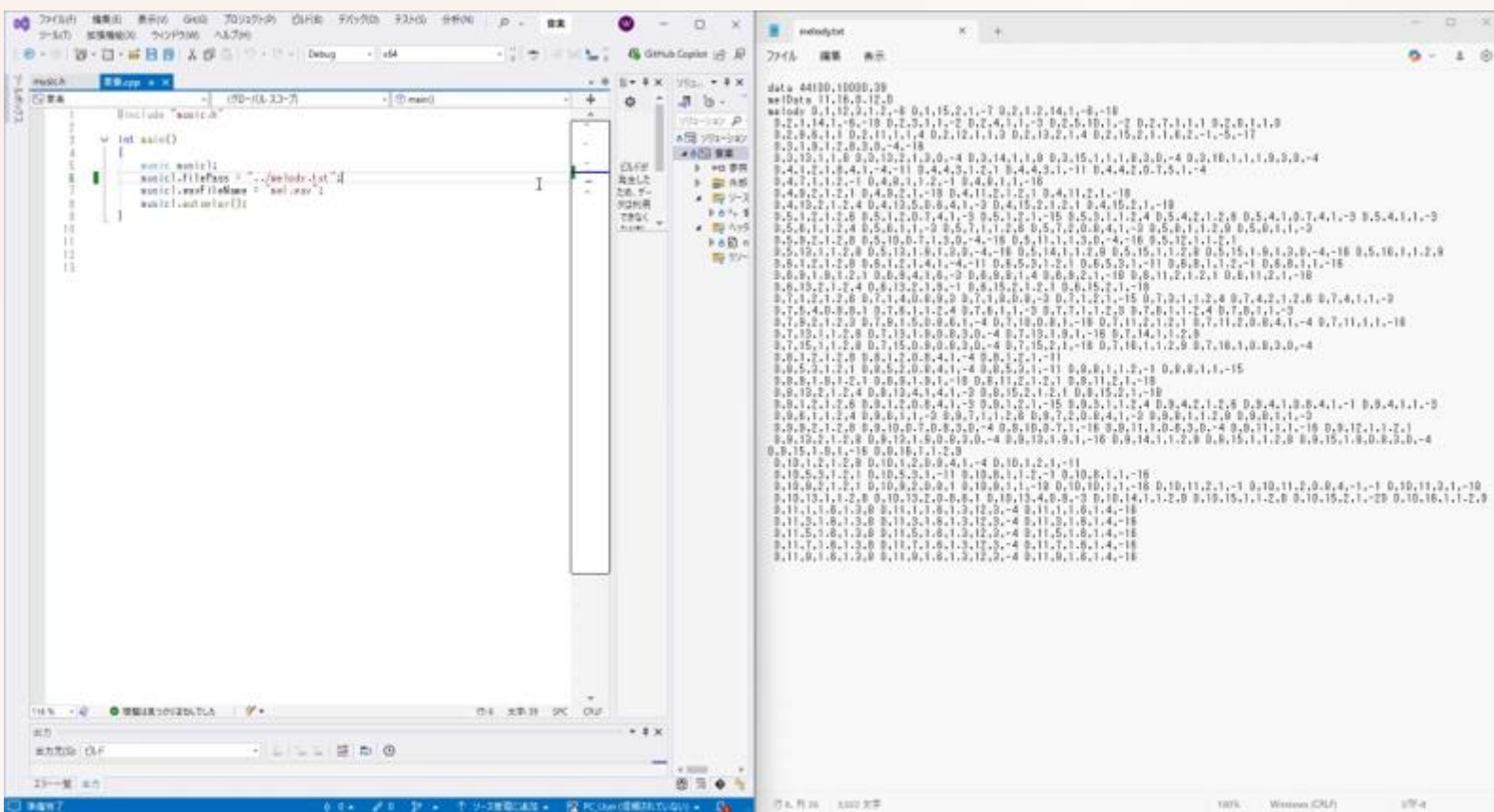
▲.wavファイルのフォーマット部です。

pythonで作成した際には、かなり楽に音を再生する機能(ライブラリ)がありましたが、C++では使えませんでした。

そこで、自力で音楽ファイル(.wav)を生成し、windowsの標準機能を使って再生するという手段をとりました。

wavファイルはなかなか難解で、16進数のみで構成されていたり、音データの前に決まったフォーマットで情報を載せなければいけませんでした。

↓の動画の右側の数字たちは、手で入力した楽譜(?)です。これをもとに音楽を奏でます。ぷつぷつ音はご愛嬌です。



▲動画です (00:27) 音が鳴ります。クリックでyoutubeに遷移し動画が見れます。
(<https://youtu.be/kCIBaVV66x0?t=180>)

バウンドする点

計算としては、光点の位置、速度を保存しておき、ボタンに合わせて加速度を変更しています。

壁に当たった際は、壁方向ベクトルをいじって反射させていますが、たまにめり込むことがあります。
動画中でも実は壁にめりこんでいます。
ゲームのバグなどにかかなりの既視感を覚えました。

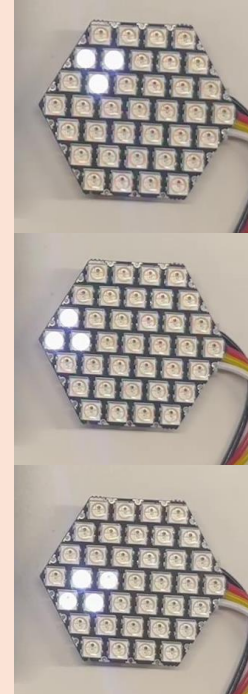
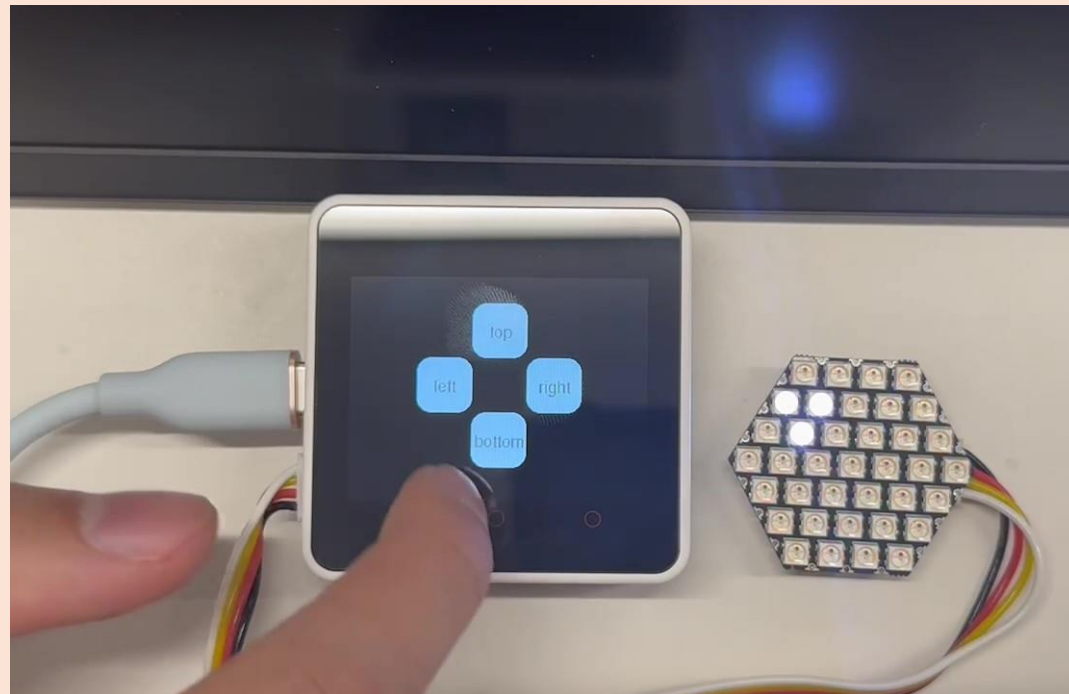
軽量化の工夫として、全体を4エリアに分け、該当するエリア内の壁の当たり判定のみチェックするようにしています。この程度の重さの処理ですとあまり意味はありませんが、ゲームエンジンに似た仕組みを作ってみたくて実装しました。

これ全体で320行ほどのコードです。
下のコードなどを含め、1週間ほどで開発しました。

右は、
「LED パネルに（グラデーションのような）動きのある表示パターンをさせる機能」というお題で作成したものです。
1秒で1周します。

C++も初めてでしたし、マイコンを使うのも初めてだったので苦戦しましたが、綺麗なコードか組めたと思います。

今思い返すと、これでもライブラリという機能でかなり簡単になっていたなと感じます。



▲動画です (00:33)
クリックでyoutubeに遷移し動画が見れます
(<https://youtu.be/kCIBaVV66x0?t=209>)

Arduino 2025年3月
制作期間 1週間
C++

研究室にてC++の勉強会の際、作成したものです。
M5Stackというマイコンを使用し、光点がバウンドするプログラムを組みました。

初めてC言語系列に触れたころのものです。

こだわった点

先生から課された課題に対し、追加で色々機能を盛り込みこうなりました。

小学校のころExcelのVBAの授業にて、壁で反射するコードを書いた経験を思い出しながらのコーディングになりました。斜めの壁がとても厄介でした。



ロボットカー



Python 2023年7月
制作期間 半期
Jupyter Notebook
授業内で優勝しましたいえい

応用Pythonという授業にて、半年かけてグループでコード開発を行いました。

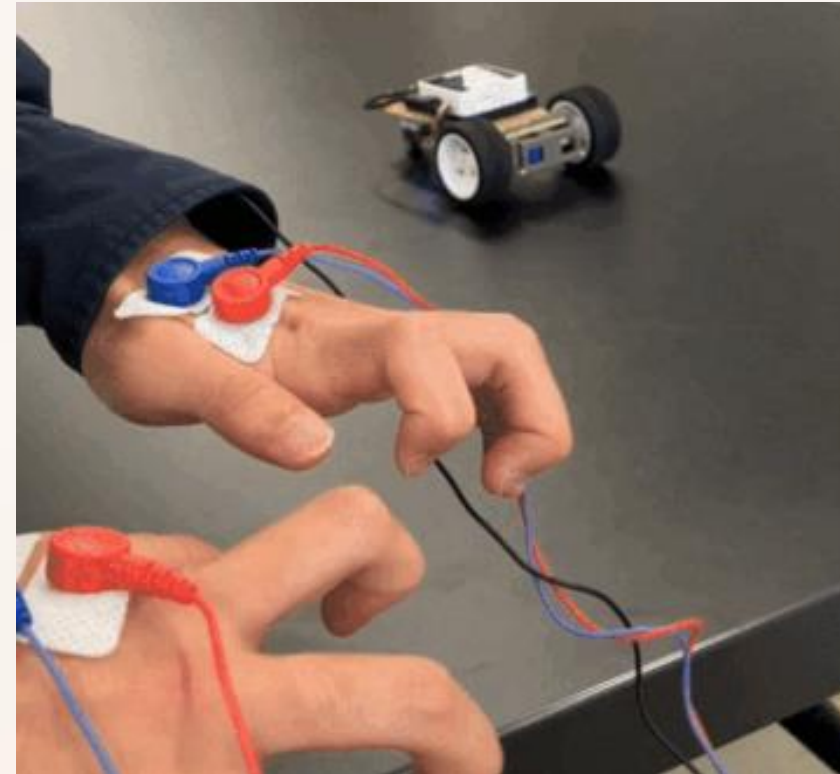
最後の授業では、各班の作成したコードを取り込んだロボットカーでロボットカーコンテストを行いました。

左右の手の筋肉の信号を読み取って、それでロボットカーを左右に曲げたりといった制御をします。

こだわった点

操作が重くなりすぎず、また今どんな数値が入っているのかわかりやすいように工夫しました。

また、操作感に個人差が出ないよう、最大のパワーを測ってフィットさせてくれるプログラムなどの制作も担当しました。なおタイムアタックで急いていたので、測るのに10秒弱くらいかかるこれは1度も使いませんでした。悲しい。



課題はリモコンカーをリアルタイムに制御するというものです。コードの構造や工夫がもろに操作感に表れるので、変数を調整するだけでも大変でした。

みんなで相談やトライアンドエラーをしながら、少しずつコードを改善していきました。

二時間連続の授業でかなり骨太な授業でしたが、かなり楽しくグループ開発を経験できました。

研究室はこの授業がきっかけとなり、これを担当してくださった先生の研究室に配属しました。

実行環境はJupyter Notebookです。

私が担当した箇所は、チームで作ったコードをつなぎ合わせる作業や全体フロー、その他細かいバク取りなどでした。

改良案 (ローパスの精度ダウン、代わりに計算速度アップ、またシリアル通信のデータの破損によるエラーを排除)

シリアル通信によるデータ送信の際、例えば "0.0.0.0Vn" が "0.0.0Vn" のように欠損して送られた結果エラーを吐き、コードが止まることなどがあったので、データ欠損時にはデータを0.0に置き換えるようにしました。

```
[6]: # 補正あり操作用 無限
e = 0

ScrIP = "192.168.247.173"
DstIP = "192.168.247.209"
socketStart(ScrIP, DstIP)

n = 25

ldata = [0]*60 #こここの数字が変わってる。いろいろ変えてみて
Rdata = [0]*60
N = 6
Wn = 8
sos = signal.butter(N, Wn, 'lowpass', output='sos', fs=500)

if "LC" in globals():
    if LC != 12 and RC != 12:
        print("補正値が発見されました")
    else:
        LC = 12
        RC = 12
        print("補正値 右 ", RC, " 倍、左 ", LC, " 倍")

ser = serial.Serial("/dev/ttyUSB0", 115200, timeout=1000)
while True:
    for b in range(0,n):
        try: # 1つ目の処理
            data = ser.readline().rstrip().decode('utf-8', 'ignore').split(",")
            ldata.append(float(data[1])*LC)
            Rdata.append(float(data[0])*RC)
        except: # tryのなかでエラーが起きたらこっちに飛ぶ
            ldata.append(0.0)
            Rdata.append(0.0)
            e += 1
    del ldata[:n]
    del Rdata[:n]
    Rf = signal.sosfiltfilt(sos, Rdata)
    Lf = signal.sosfiltfilt(sos, ldata)
    order = str(Lf[-1])+"-"+str(Rf[-1])
    udpClientSock.sendto(order.encode('utf-8'), DstAddr)
    print("\n", " エラー発生 ", e, " 件", order, end='')

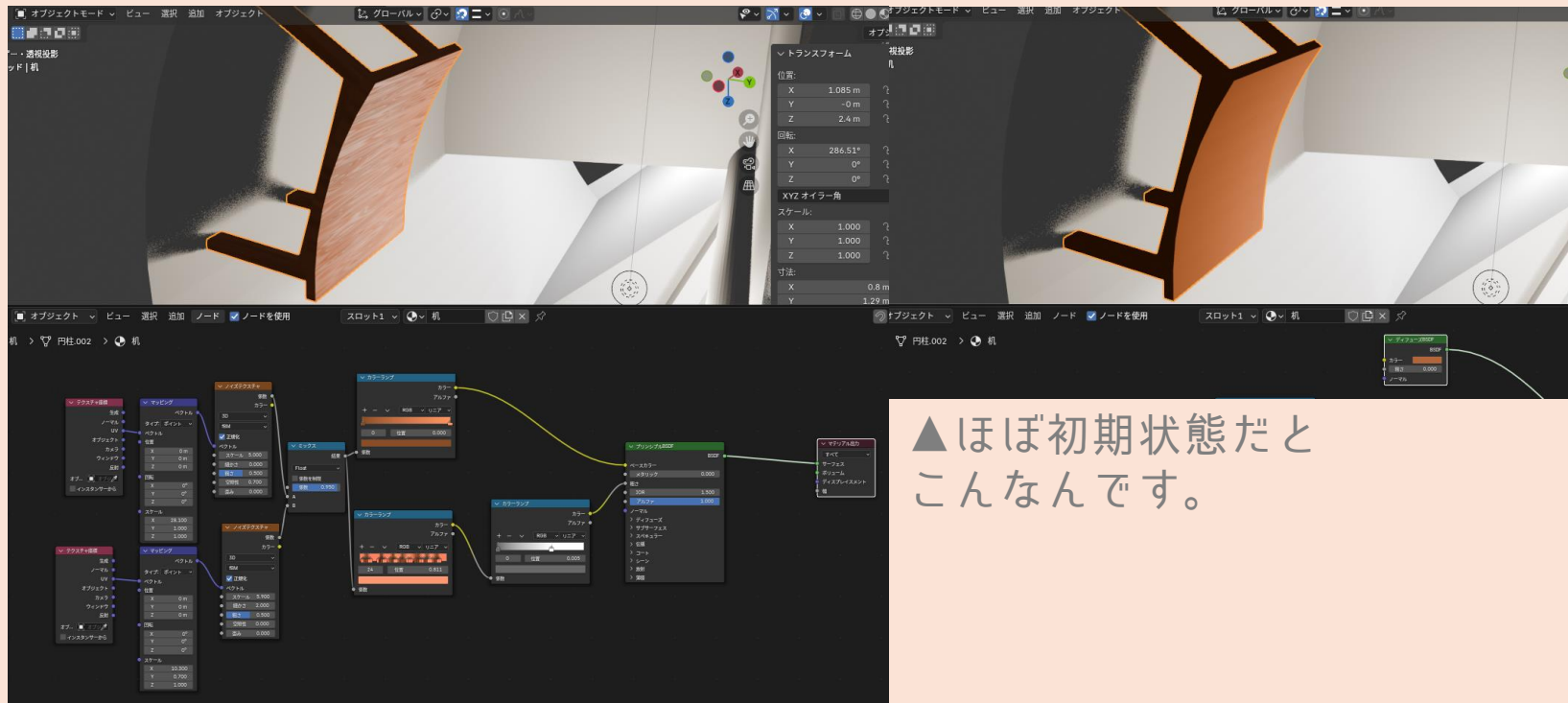
補正値 右 12 倍、左 12 倍
中断したあとに実行してください。通信を終了します。

[7]: order = "0,0"
udpClientSock.sendto(order.encode('utf-8'), DstAddr)
socketClose()
ser.close()

[10]: print(LC, RC)

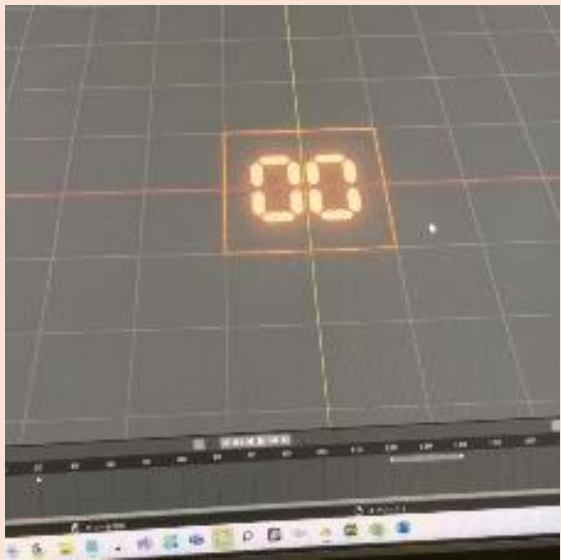
5.048913888835006 4.19112088970995
```

その他

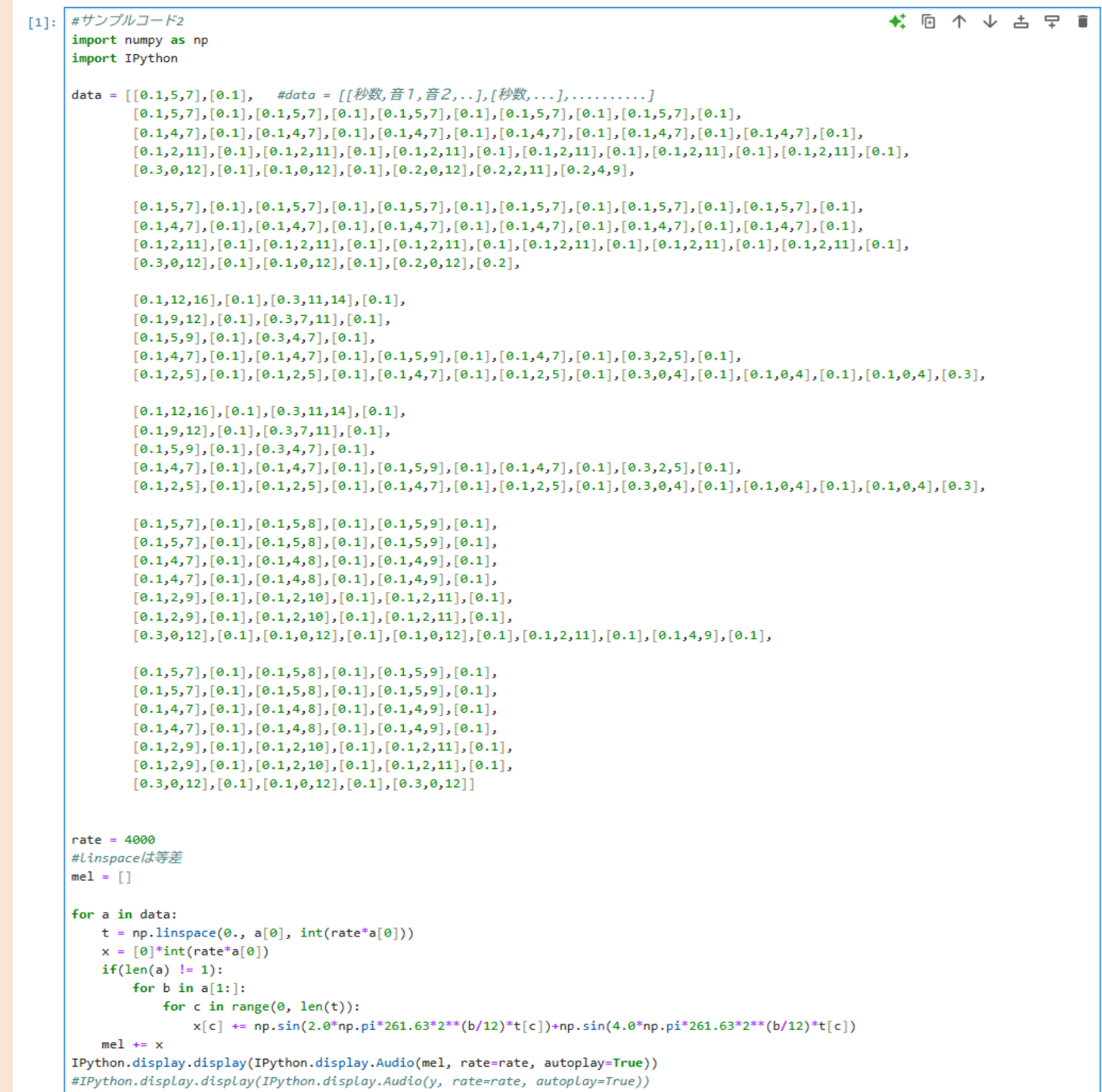


▲blenderでノードプログラミングはよく組めます。こちらは机の木の表面をノードで設定したものです。なんとなくで年輪を意識して組んでみましたが、もっと実力をつけて理論的かつ良い感じにやりたいです。

重力が変な世界観で作っていたので、
部屋の形やらが、すごいことになっています。



◀同じくblenderのノードです。
0~100までの整数値を入力すると
デジタル数字で出力されます。



2023年7月

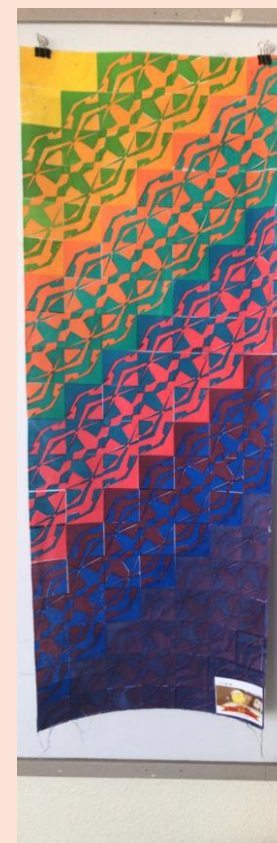
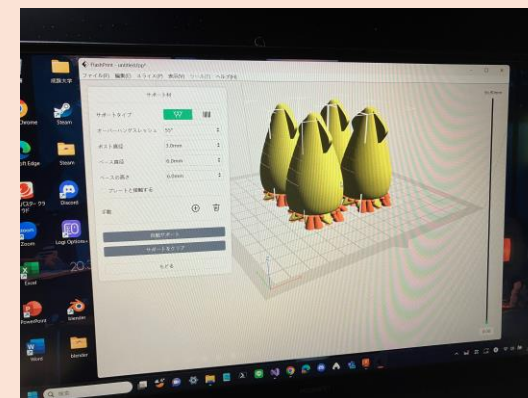
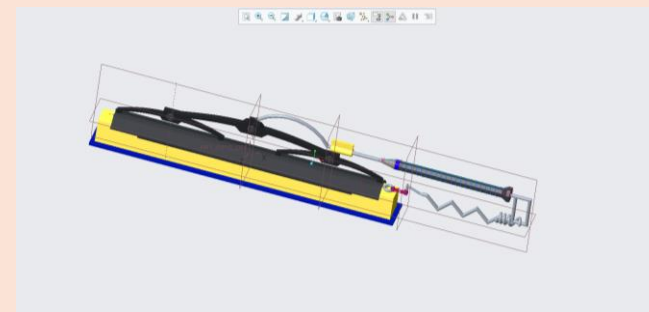
▲音楽再生コードの基礎部分になったものです。(Python)
sin波の重ね合わせで配列に音の高さを挿入しています。
wavファイルの生成はライブラリに任せている簡易版です。

ゲーム開発

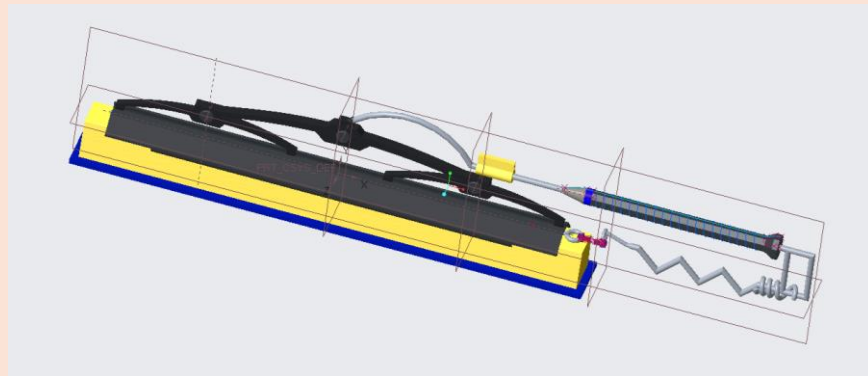
3DCD

プログラミング

その他
(CADと絵)



全2ページ



▲中間課題_ドライブワイパー



▲期末課題_スパイガシェット

この2つは1年のCADの授業の課題で作ったものです。自由なものを作れとのことでしたので、スプラトゥーンシリーズの武器を作成しました。
どちらもお手本をよく見てしっかり再現することにこだわり、数式で立体の曲線を描いたり、開閉のギミックを搭載しました。

2年のCADの授業では先生から扇風機を渡され、グループで分担してCADで再現しろとのことでした。

私の担当は首振り機構や羽、ギアボックスでした。きちんと首を振る様を見たときには達成感がすごかったです。



▲全体



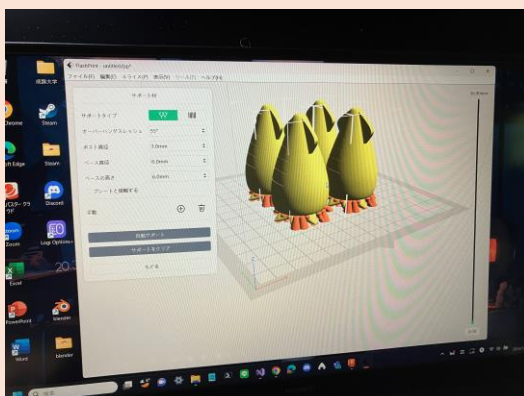
▲担当した箇所全部

CAD

こだわったポイント

3Dプリンターを授業で使い、先生に頼み込んで個人の印刷もいくつか経験させて頂きました。

これは机の上に置物として設置しようと置物を出力したものです。なかなか難しいものです。



▲CAM出力確認画面



▲残念な出力結果



◀裏側



◀取り付け

テンキーの位置にマウスを置けたら楽だな… と思い、キーボードを計測して作成しました。ピッタリサイズです。

ここまでが2023年までのCADの経験です。この経験もあって、2024年頭にPCを購入してblenderをやり始めました。

絵

中高で美術の授業が楽しかった記憶があり、大学では美術部に入部しました。
blenderを始める前までは描いた絵などを展示会などに出していました。

▲「絵」



▲「ぼかし」

高校の授業で作成したものです。
シルクスクリーンという技法です。
地味に二種類の型紙しか使って
いませんが、代わりに170回ほど
刷りました。

べた塗りでグラデーションをして
みたくて、調整を重ねました。

鉛筆が好きなので、全て鉛筆で描き
ました。
モデルは大学がある吉祥寺のアパート
で、昼に撮った写真から夜を想像して
描きました。
このために6Bの鉛筆を買いました。

▼「夜」



▲「街」

アクリル絵の具で描きました。
「夜に見えるけど実は深い海底に沈んだ街
で、少数の人間が適応して暮らしている」
いう設定のつもりでしたが、水感を出すのが難
しすぎました。普通に夜に見えます。
ですが、色味がかなり良い感じにいったと
思います。お気に入りです。

デジタルになれなくて、かつ友達に勧めら
れたインディーゲームに登場する食べ物の絵
に憧れて作りました。スマホで描いています。
魅力的な絵を書くのは難しいなと痛感させ
られました。
ちなみに「勇者の飯」というスマホゲーム
の絵に憧れました。本当においしそうなお絵が
登場します。

▼「フロランタン」

